VIDEOZEN – Complete Feature Summary Table

Table: Features Implemented vs Improvements Added

Feature	Implemented in Code?	Explanation
Hybrid Encryption System	**YES**	Blowfish + AES layered encryption implemented.
Blowfish Encryption Layer	**YES**	Inner layer encryption on every frame.
AES Encryption Layer	**YES**	Outer layer encryption applied after Blowfish.
Dual-layer Security	**YES**	Encryption flow: Blowfish → AES.
Protection Against Brute Force Attacks	**YES**	Two keys + two IVs per frame drastically increase complexity.
Protection Against Differential & Linear Cryptanalysis	**YES**	AES + Blowfish hybrid improves resistance.
Video Security for Transmission/Storage	**YES**	Encrypted frames cannot be viewed without keys.
Integrity & Confidentiality	**YES**	CBC mode + secure keys ensure both.
Python-based Implementation	**YES**	Complete project implemented using Python.
Use of OpenCV for Video Processing	**YES**	Used for frame extraction & video reconstruction.
Use of Perceptome	**NO**	PyCryptodome used instead of Perceptome.
Securing Videos for DRM/E-learning/Surveillance	**YES**	Architecture supports use cases indirectly.
Layered Security Architecture	**YES**	Proper separation of components & folders.
Efficient & Fast Encryption	**YES**	Blowfish is lightweight; AES is optimized.
Tamper-Proof Video Sharing	**YES**	Without keys, video cannot be reconstructed.
Unauthorized Access Prevention	**YES**	Requires exact Blowfish + AES keys to decrypt.
Multimedia Security Enhancement	**YES**	Protects both video frames & audio.
High-Level Confidential Communications Support	**YES**	Strong encryption assures confidentiality.
Easy Integration into Applications	**YES**	Modular and clean architecture.
Frame-level encryption	**EXTRA**	Each frame is encrypted individually for better security.

Feature	Implemented in Code?	Explanation
Random IV per frame (AES + Blowfish)	**EXTRA**	IV varies for each frame, preventing pattern attacks.
Audio extraction	**EXTRA**	Audio is separated and secured.
Rebuild video with audio	**EXTRA**	Combines decrypted frames + audio to form final video.
Structured storage (frames, encrypted, decrypted, keys, audio)	**EXTRA**	Professional folder organization for each project.
Original video preview in UI	**EXTRA**	User can play uploaded video in browser.
Output video preview	**EXTRA**	Decrypted video displayed in UI.
Flash-based success/error messages	**EXTRA**	Clearly shows process status in UI.
Delete upload/project	**EXTRA**	Removes all related files/folders safely.
Enhanced error handling	**EXTRA**	Displays clean console logs and flash messages.
Responsive & animated UI (HTML/CSS)	**EXTRA**	Improves usability and user experience.

🟦 Videozen Pro – Simple Explanation of How the Code Works

Videozen Pro is a system that **protects videos** by breaking them into pieces (frames), encrypting them, and rebuilding them back again. It uses **strong dual-layer encryption** so no one can view or modify the video without the correct keys.

Below is a clear simple explanation of each step.

🔵 1. Uploading a Video

- The user uploads a video file.
- The system creates a new folder for that video so all its files stay organized.
- The uploaded video is saved safely in the original/ folder.

Purpose:
To organize the project and store the original file.

🔵 2. Extracting Audio

- The code takes the audio out from the original video.
- The audio is saved as audio.mp3.

Purpose:
To keep the original sound so it can be added back later.

🔵 3. Splitting Video into Frames

- The code breaks the video into many images (frames).
- Each frame is saved as a .jpg image.

Example:

frame_000000.jpg

frame_000001.jpg

...

Purpose:
Encrypting frames separately makes the video very secure, because even one missing frame makes the video useless.

🔵 4. Generating Encryption Keys

- The system creates two random files:
  - bf.key → Blowfish key
  - aes.key → AES key
- These keys are saved in the keys/ folder.

Purpose:
To lock (encrypt) and unlock (decrypt) the video frames.

🔵 5. Encrypting the Frames

Each frame goes through **two encryption steps**:

1. **Blowfish encryption (inner layer)**
   Scrambles the raw image.
2. **AES encryption (outer layer)**
   Scrambles the Blowfish output again.

- A small file called .enc is created for every frame.

Purpose:
To make the video unreadable and fully secure.

Without both keys, frames cannot be restored.

🔵 6. Decrypting the Frames

- The encrypted frames are read back.
- AES is removed first.

- Blowfish is removed second.
- The original frame image is restored.

Purpose:
To recover the original frames safely.

🔵 7. Rebuilding the Video

- All decrypted frames are combined back into a video file using OpenCV.
- The saved audio (audio.mp3) is added back.
- The completed video is stored as output_final.mp4.

Purpose:
To rebuild the original playable video.

🔵 8. Viewing or Downloading Output Video

- The final video is shown in the browser.
- The user can download it or watch it.

Purpose:
To view the result of the secure encryption/decryption cycle.

🔵 9. Deleting the Project

- The user can delete the entire folder.
- All frames, keys, audio, and output files are deleted.

Purpose:
To protect privacy and free storage.

⭐ Purpose of the Entire System

✓ To securely protect video content

✓ To prevent unauthorized access

✓ To ensure video cannot be viewed or reconstructed without the keys

✓ To support safe storage, sharing, and transmission

✓ To provide a complete encryption + decryption workflow